Tmux 使用教程

Geoffrey Xu
13149131068@163.com
Xidian University

March 4, 2026

Abstract

本文面向首次使用 Tmux 的读者，系统介绍会话（session）、窗口（window）、窗格（pane）等核心概念，并给出常用命令、快捷键、配置示例、复制粘贴与插件方案，帮助你在本地开发与远程 SSH 场景下稳定地「保护现场」并高效管理终端工作流。

Keywords tmux · terminal multiplexer · session · window · pane · SSH

目录

1. 概述 3
2. 快速开始 4
   2.1. 安装 4
   2.2. 启动与退出 4
   2.3. 前缀键（Prefix） 5
3. 核心概念 5
4. 会话管理（Session） 6
   4.1. 新建会话 6
   4.2. 分离会话（Detach） 6
   4.3. 查看会话列表 7
   4.4. 接入会话（Attach） 7
   4.5. 切换会话（Switch） 8
   4.6. 重命名会话 8
   4.7. 杀死会话 8
   4.8. 最简操作流程 8
   4.9. 会话相关快捷键（汇总） 9
5. 窗口管理（Window） 9
   5.1. 新建窗口 9
   5.2. 切换窗口 9
   5.3. 重命名窗口 9
   5.4. 窗口快捷键（常用） 9
6. 窗格操作（Pane） 10
   6.1. 划分窗格 10

6.2. 移动光标（选择窗格） 10
6.3. 交换窗格位置 11
6.4. 窗格快捷键（常用） 11
7. 快捷键速查表（表格版） 11
7.1. 系统指令 11
7.2. 窗口（window）指令 12
7.3. 面板（pane）指令 12
8. 配置入门（ /.tmux.conf） 13
8.1. 修改前缀键 13
8.2. 把分屏键改得更直观 14
8.3. 开启鼠标支持 14
8.4. 快速窗格切换：绑定 hjkl 14
8.5. 面板大小调整：绑定 Ctrl+hjkl 15
8.6. 面板最大化（Zoom）与兼容方案 15
8.7. 窗口变为面板：join-pane 16
8.8. 其他配置示例 16
9. 复制模式、Buffer 与系统粘贴板 16
9.1. 复制模式的基本流程 16
9.2. 自定义复制与选择快捷键 17
9.3. Buffer 缓存 17
9.4. Linux：接入系统粘贴板（xclip） 18
9.5. macOS：接入系统粘贴板（pbcopy / pbpaste） 18
10. 会话持久化：Resurrect / Continuum / TPM 19
10.1. Tmux Resurrect 20
10.2. Tmux Continuum 21
10.3. TPM（Tmux Plugin Manager） 22
11. 会话共享（结对编程 / 远程教学） 23
11.1. 使用 tmate 快速分享 23
11.2. 组会话（共享账号场景） 24
11.3. Socket 共享（不同账号场景） 24
12. 优化与脚本化实践 25
12.1. 窗口/面板起始编号 25
12.2. 状态栏优化（示例） 25
12.3. 256 colors 支持 26
12.4. 关闭自动重命名（节省资源） 26
12.5. 去掉小圆点（避免窗口被最小尺寸限制） 26
12.6. 脚本化创建工作环境（示例） 27
12.7. macOS：开机自动启用脚本 28
13. 其他命令 28
14. Reference 29

1. 概述

Tmux 是一种终端复用器（terminal multiplexer）。它把“终端窗口（window）”与其中运行的“会话（session）/进程”解耦：关闭终端窗口时，会话可以继续在后台运行；之后你可以再把一个新的终端窗口接入（attach）到原会话中，继续工作。这对远程 SSH、长时间运行的任务、以及需要并行多个命令行程序的开发场景尤为重要。

Tmux 的典型能力包括：

- 在单个终端窗口里同时管理多个会话/窗口/窗格。
- 断开（detach）后保持现场，随时重新接入（attach）。
- 允许同一会话被多个客户端连接，从而实现多人共享（适合结对编程/远程教学）。
- 支持水平与垂直分屏，并进行窗格（pane）的切换、交换与调整大小。

典型使用场景：

- 你希望桌面/终端“更简洁”：减少大量终端窗口与浏览器标签页带来的注意力分散。
- 你需要同时跑多个命令行程序：日志、构建、调试、数据库客户端等，需要分屏并同时观察输出。
- 你经常 SSH 到远端跑长任务：网络抖动、断线重连都不应影响任务继续运行。
- 你希望“静默无感知”地准备环境：开机后自动启动本地多个服务，但不想弹出大量窗口；关闭终端后现场仍然保留，随时可接入继续操作。

为什么选择 Tmux（三个关键点）：

- 丝滑分屏（split）：相比某些终端模拟器的分屏，tmux 新窗格通常会继承当前路径；如果处于 SSH，会话状态也能继续保持，减少重复登录/切目录的操作成本。
- 保护现场（attach）：关闭终端或断线后仍可回到现场，尤其适合远端耗时任务。
- 会话共享：把会话“入口”分享出去，其他人可通过 SSH 进入同一会话，适用于结对编程或远程教学。

类似工具还有 GNU Screen。相比之下，Tmux 的快捷键体系与配置机制更现代、可扩展性更强。

图 1　Tmux 示例界面（截图）

2. 快速开始

2.1. 安装

下面给出常见系统的安装方式（命令行中执行）。

```
# Ubuntu 或 Debian
$ sudo apt-get install tmux

# CentOS 或 Fedora
$ sudo yum install tmux

# Mac
$ brew install tmux
```

如果你在 macOS 上还没有 Homebrew，可以参考下面“先安装 Homebrew、再安装 tmux”的示例（Homebrew 的安装方式可能随时间变化，此处仅作参考）：

```
# 先安装Homebrew，有则跳过
ruby -e "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/
master/install)"
# 安装tmux
brew install tmux
```

2.2. 启动与退出

安装后，直接运行 `tmux` 即可进入 Tmux。

```
$ tmux
```

进入后，你会看到底部的状态栏：左侧通常是窗口编号与名称，右侧是系统信息。

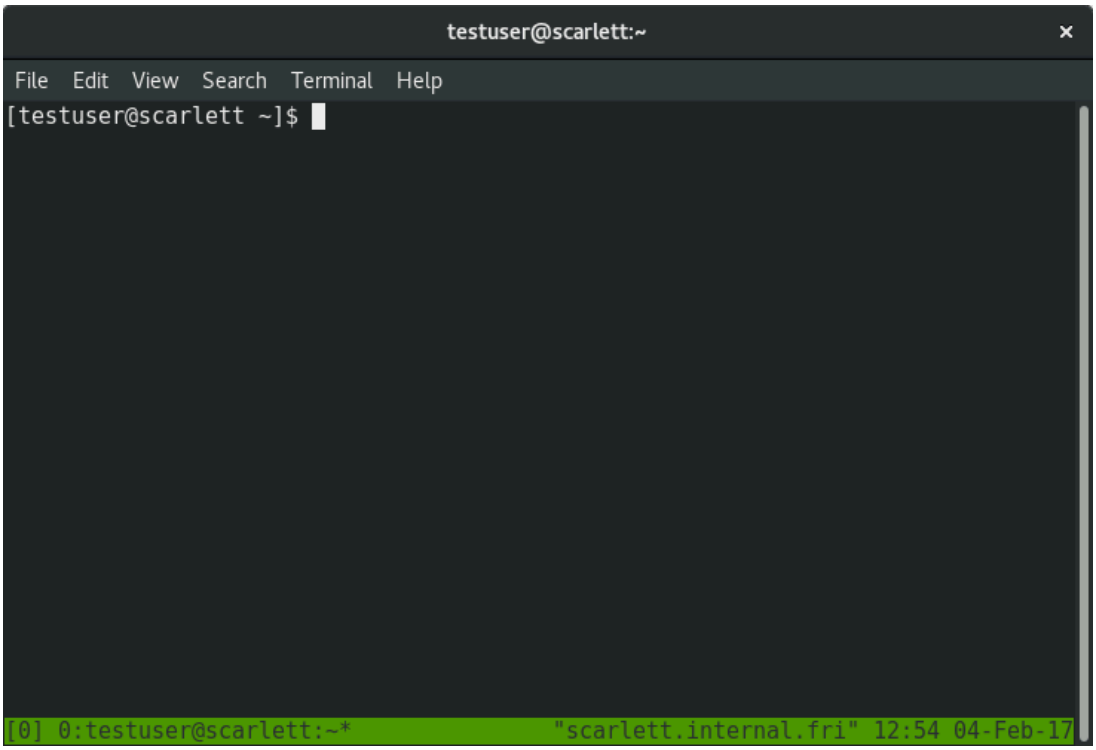

图 2　状态栏示例（截图）

退出 Tmux 的方式：

- 直接按 `Ctrl+d`（结束当前 shell）。
- 或显式输入 `exit`。

```
$ exit
```

2.3. 前缀键（Prefix）

Tmux 的快捷键都以“前缀键”开头。默认前缀键是 `Ctrl+b`：先按 `Ctrl+b`，再按后续按键，指令才会生效。

一个最常用的帮助指令是 `Ctrl+b ?`：显示帮助后，按 `Esc` 或 `q` 退出帮助界面。

3. 核心概念

在开始大量快捷键之前，建议先统一术语：

- 会话（session）：一组持久存在的终端工作环境。你可以断开并重新接入。
- 窗口（window）：会话中的“标签页”，一个会话可包含多个窗口。
- 窗格（pane）：窗口内的分屏区域，一个窗口可包含多个窗格。

实践上可以这样理解两者的分工：

- 一个会话里的多个窗口通常“互相独立”，适合放相关性不强的任务（例如一个窗口跑后端、另一个窗口跑前端、第三个窗口做运维）。
- 同一窗口里的多个窗格处在同一屏幕内，适合放相关性强、需要同时观察的任务（例如编译输出 + 服务日志 + REPL）。

可以用“C/S 模型”理解 tmux：执行 `tmux` 相当于启动服务器；默认创建一个会话，会话默认创建一个窗口，窗口默认创建一个窗格。关系如下图所示。

图 3　session / window / pane 的关系（示意图）

4. 会话管理（Session）

4.1. 新建会话

如果只是运行 `tmux`，会创建一个未命名会话。为了便于管理，建议显式命名：

```
$ tmux new -s <session-name>
```

会话入门命令（示例）：

```
tmux # 新建一个无名称的会话
tmux new -s demo # 新建一个名称为demo的会话
```

第一次启动的 Tmux 会话常编号为 `0`，第二个为 `1`，以此类推；但用编号管理不直观，因此更推荐命名会话。

在脚本/自动化场景中，可以使用 `-d` 在后台创建会话：

```
tmux new -s init -d # 后台创建一个名称为init的会话
```

4.2. 分离会话（Detach）

在会话中按 `Ctrl+b d`，或者执行 `tmux detach`，即可断开当前会话与终端窗口的绑定。

```
$ tmux detach
```

下面给出同一操作的简洁示例（便于记忆）：

```
tmux detach # 断开当前会话，会话在后台运行
```

断开后会话仍在后台运行，里面的进程不会因为你关闭终端而终止。

4.3. 查看会话列表

```
$ tmux ls
# or
$ tmux list-session
```

下面给出对应命令（并建议使用简写）：

```
tmux list-session # 查看所有会话
tmux ls # 查看所有会话，提倡使用简写形式
```

如果你正处于会话内部，还可以使用 `Ctrl+b s` 打开会话列表：上下键（⬆⬇）或鼠标滚轮选择目标；左右键（⬅➡）收起/展开会话下的窗口；回车完成切换。

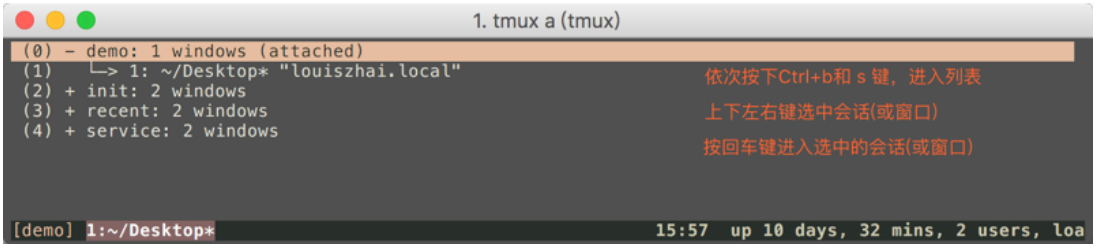

图 4　会话列表界面（截图）

4.4. 接入会话（Attach）

```
# 使用会话编号
$ tmux attach -t 0

# 使用会话名称
$ tmux attach -t <session-name>
```

此外还有简写：`tmux a`（默认进入第一个会话），以及 `tmux a -t demo`（进入指定会话）。

```
tmux a # 默认进入第一个会话
tmux a -t demo # 进入到名称为demo的会话
```

4.5. 切换会话（Switch）

```
# 使用会话编号
$ tmux switch -t 0

# 使用会话名称
$ tmux switch -t <session-name>
```

4.6. 重命名会话

```
$ tmux rename-session -t 0 <new-name>
```

4.7. 杀死会话

```
# 使用会话编号
$ tmux kill-session -t 0

# 使用会话名称
$ tmux kill-session -t <session-name>
```

补充说明：“kill”家族常见成员包括 `kill-pane`、`kill-window`、`kill-session`、`kill-server`。其中 `kill-server` 会关闭服务器并终止所有会话。

```
tmux kill-session -t demo # 关闭demo会话
tmux kill-server # 关闭服务器，所有的会话都将关闭
```

4.8. 最简操作流程

如果你只想先跑通一遍“保护现场”的核心流程，可以按下面四步走：

1. 新建会话（建议命名，便于后续接入）。
2. 在会话里运行需要的程序。
3. 分离会话（detach），让进程继续在后台跑。
4. 下次再接入会话（attach），回到原现场继续工作。

```
tmux new -s my_session
```

（此处在 tmux 窗口中运行你的程序）

```
Ctrl+b d
```

```
tmux attach-session -t my_session
```

4.9. 会话相关快捷键（汇总）

会话相关快捷键（常用）：
- `Ctrl+b d`：分离当前会话。
- `Ctrl+b s`：列出所有会话。
- `Ctrl+b $`：重命名当前会话。

5. 窗口管理（Window）

5.1. 新建窗口

```
$ tmux new-window

# 新建一个指定名称的窗口
$ tmux new-window -n <window-name>
```

5.2. 切换窗口

```
# 切换到指定编号的窗口
$ tmux select-window -t <window-number>

# 切换到指定名称的窗口
$ tmux select-window -t <window-name>
```

5.3. 重命名窗口

```
$ tmux rename-window <new-name>
```

5.4. 窗口快捷键（常用）

- `Ctrl+b c`：创建新窗口。
- `Ctrl+b p`：切换到上一个窗口。

- `Ctrl+b n`：切换到下一个窗口。
- `Ctrl+b <number>`：切换到指定编号窗口。
- `Ctrl+b w`：从列表选择窗口。
- `Ctrl+b ,`：重命名窗口。

6. 窗格操作（Pane）

6.1. 划分窗格

```
# 划分上下两个窗格
$ tmux split-window

# 划分左右两个窗格
$ tmux split-window -h
```

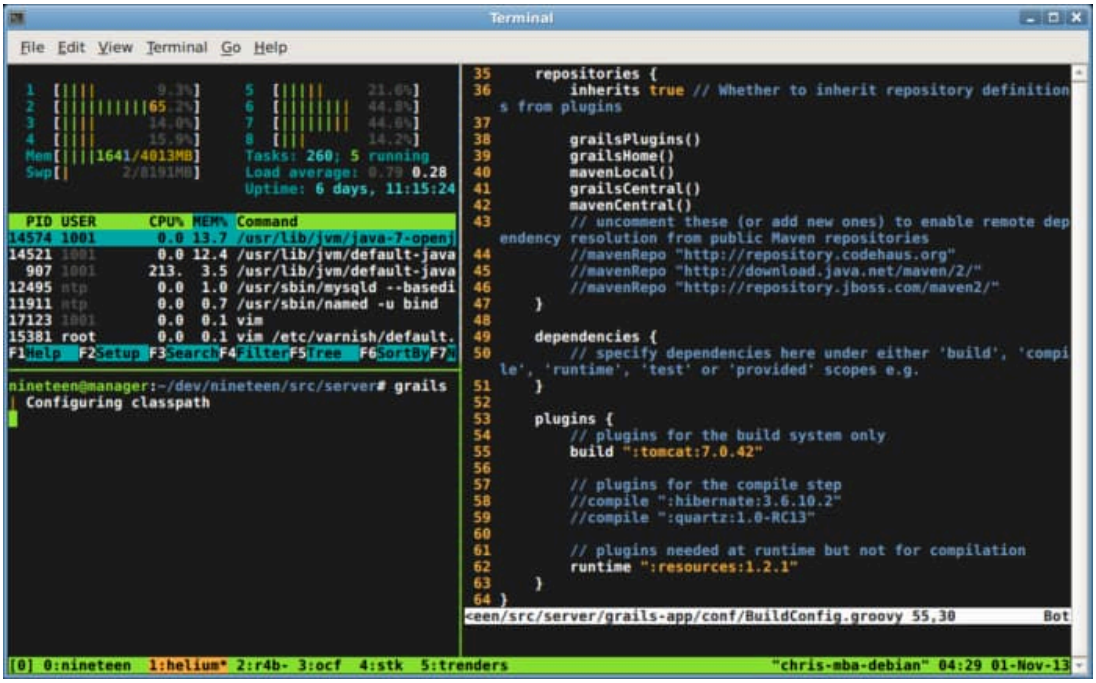

图 5　分屏示例（截图）

6.2. 移动光标（选择窗格）

```
# 光标切换到上方窗格
$ tmux select-pane -U

# 光标切换到下方窗格
$ tmux select-pane -D
```

```
# 光标切换到左边窗格
$ tmux select-pane -L

# 光标切换到右边窗格
$ tmux select-pane -R
```

6.3. 交换窗格位置

```
# 当前窗格上移
$ tmux swap-pane -U

# 当前窗格下移
$ tmux swap-pane -D
```

6.4. 窗格快捷键（常用）

- `Ctrl+b %`：左右分屏。
- `Ctrl+b "`：上下分屏。
- `Ctrl+b <arrow key>`：用方向键在窗格间移动光标。
- `Ctrl+b ;`：切换到上一个窗格。
- `Ctrl+b o`：切换到下一个窗格。
- `Ctrl+b {`：与上一个窗格交换位置。
- `Ctrl+b }`：与下一个窗格交换位置。
- `Ctrl+b Ctrl+o`：所有窗格向前移动一个位置。
- `Ctrl+b Alt+o`：所有窗格向后移动一个位置。
- `Ctrl+b x`：关闭当前窗格。
- `Ctrl+b !`：将当前窗格拆分为独立窗口。
- `Ctrl+b z`：窗格最大化/还原。
- `Ctrl+b Ctrl+<arrow key>`：按方向键调整窗格大小。
- `Ctrl+b q`：显示窗格编号。

7. 快捷键速查表（表格版）

下面三张表把常用快捷键按“系统 / 窗口 / 面板”分组整理（默认前缀均为 `Ctrl+b`）。如果你修改了前缀键，请把表中的 `Ctrl+b` 等价替换为你的前缀键。

7.1. 系统指令

前缀	按键	描述
`Ctrl+b`	`?`	显示快捷键帮助文档

`Ctrl+b`	`d`	断开当前会话
`Ctrl+b`	`D`	选择要断开的会话
`Ctrl+b`	`Ctrl+z`	挂起当前会话
`Ctrl+b`	`r`	强制重载当前会话
`Ctrl+b`	`s`	显示会话列表用于选择并切换
`Ctrl+b`	`:`	进入命令行模式，可直接输入 `ls` 等命令
`Ctrl+b`	`[`	进入复制模式，按 `q` 退出
`Ctrl+b`	`]`	粘贴复制模式中复制的文本
`Ctrl+b`	`~`	列出提示信息缓存

7.2. 窗口（window）指令

前缀	按键	描述
`Ctrl+b`	`c`	新建窗口
`Ctrl+b`	`&`	关闭当前窗口（关闭前需输入 `y` 或 `n` 确认）
`Ctrl+b`	`0~9`	切换到指定窗口
`Ctrl+b`	`p`	切换到上一窗口
`Ctrl+b`	`n`	切换到下一窗口
`Ctrl+b`	`w`	打开窗口列表，用于切换窗口
`Ctrl+b`	`,`	重命名当前窗口
`Ctrl+b`	`.`	修改当前窗口编号（适用于窗口重新排序）
`Ctrl+b`	`f`	快速定位窗口（输入关键字匹配窗口名称）

7.3. 面板（pane）指令

前缀	按键	描述
`Ctrl+b`	`"`	当前面板上下一分为二，下侧新建面板
`Ctrl+b`	`%`	当前面板左右一分为二，右侧新建面板
`Ctrl+b`	`x`	关闭当前面板（关闭前需输入 `y` 或 `n` 确认）
`Ctrl+b`	`z`	最大化当前面板，再次按下恢复（v1.8 新增）
`Ctrl+b`	`!`	将当前面板移动到新窗口打开（需至少两个面板）
`Ctrl+b`	`;`	切换到最后一次使用的面板
`Ctrl+b`	`q`	显示面板编号；编号消失前输入数字切换

`Ctrl+b`	`{`	向前置换当前面板
`Ctrl+b`	`}`	向后置换当前面板
`Ctrl+b`	`Ctrl+o`	顺时针旋转当前窗口中的所有面板
`Ctrl+b`	方向键	移动光标切换面板
`Ctrl+b`	`o`	选择下一面板
`Ctrl+b`	空格键	在自带的面板布局中循环切换
`Ctrl+b`	`Alt+方向键`	以 5 个单元格为单位调整面板边缘
`Ctrl+b`	`Ctrl+方向键`	以 1 个单元格为单位调整面板边缘（Mac 下可能被系统快捷键覆盖）
`Ctrl+b`	`t`	显示时钟

8. 配置入门（ /.tmux.conf）

Tmux 支持类似 Vim 的高度可配置性。常见的入门需求包括：修改前缀键、让分屏更顺手、启用鼠标、优化切换窗格等。

8.1. 修改前缀键

默认 `Ctrl+b` 距离较远，你可以换成更顺手的 `Ctrl+a`，并可配置第二前缀键（tmux v1.6 起支持 `prefix2`）。

```
set -g prefix C-a #
unbind C-b # C-b即Ctrl+b键，unbind意味着解除绑定
bind C-a send-prefix # 绑定Ctrl+a为新的指令前缀

# 从tmux v1.6版起，支持设置第二个指令前缀
set-option -g prefix2 ` # 设置一个不常用的`键作为指令前缀，按键更快些
```

配置生效方式：

- 重启 tmux。
- 或在 tmux 内按 `Ctrl+b :` 进入命令模式，执行 `source-file ~/.tmux.conf`。

你也可以在命令行直接执行：

```
$ tmux source-file ~/.tmux.conf
```

为了随时重载配置，可以绑定快捷键 `r`：

```
# 绑定快捷键为r
bind r source-file ~/.tmux.conf \; display-message "Config reloaded.."
```

注意：在已经创建的窗口里，即使重载了新配置，旧配置也可能仍然有效（只要新配置没有覆盖旧绑定）。新建会话会直接采用最新配置。

8.2. 把分屏键改得更直观

默认分屏键需要 `"`/`%`，你可以解绑后绑定到更直观的 `-` 和 `|`，并使用 `-c '#{pane_current_path}'` 让新窗格继承当前路径。

```
unbind '"'
bind - splitw -v -c '#{pane_current_path}' # 垂直方向新增面板，默认进入当前目录
unbind %
bind | splitw -h -c '#{pane_current_path}' # 水平方向新增面板，默认进入当前目录
```

8.3. 开启鼠标支持

开启鼠标后可用鼠标切换窗格、拖拽改变大小、在状态栏切换窗口等。

tmux v2.1（2015-10-28）之前：

```
setw -g mode-mouse on # 支持鼠标选取文本等
setw -g mouse-resize-pane on # 支持鼠标拖动调整面板的大小(通过拖动面板间的分割线)
setw -g mouse-select-pane on # 支持鼠标选中并切换面板
setw -g mouse-select-window on # 支持鼠标选中并切换窗口(通过点击状态栏窗口名称)
```

tmux v2.1 及以上：

```
set-option -g mouse on # 等同于以上4个指令的效果
```

补充：有时文档会写 `set-window-option`，而 `setw` 是其别名。开启鼠标后，iTerm2 默认“鼠标选中即复制”可能需要同时按 `Alt` 才生效。

8.4. 快速窗格切换：绑定 hjkl

```
# 绑定hjkl键为面板切换的上下左右键
bind -r k select-pane -U # 绑定k为↑
bind -r j select-pane -D # 绑定j为↓
bind -r h select-pane -L # 绑定h为←
bind -r l select-pane -R # 绑定l为→
```

`-r` 表示按键可重复：在较短时间内连续按下仍然生效，适合快速切换。

你还可以加上“最后面板/最后窗口”和“交换窗格”的快捷键：

```
bind -r e lastp # 选择最后一个面板
bind -r ^e last # 选择最后一个窗口

bind -r ^u swapp -U # 与前一个面板交换位置
bind -r ^d swapp -D # 与后一个面板交换位置
```

8.5. 面板大小调整：绑定 Ctrl+hjkl

`resizep` 是 `resize-pane` 的别名。

```
# 绑定Ctrl+hjkl键为面板上下左右调整边缘的快捷指令
bind -r ^k resizep -U 10 # 绑定Ctrl+k为往↑调整面板边缘10个单元格
bind -r ^j resizep -D 10 # 绑定Ctrl+j为往↓调整面板边缘10个单元格
bind -r ^h resizep -L 10 # 绑定Ctrl+h为往←调整面板边缘10个单元格
bind -r ^l resizep -R 10 # 绑定Ctrl+l为往→调整面板边缘10个单元格
```

8.6. 面板最大化（Zoom）与兼容方案

tmux v1.8 起支持 `prefix + z` 最大化/还原面板。更早版本可用脚本模拟“放大”：通过新建窗口与交换窗格实现，再次触发则还原并关闭新窗口。

下面是文章给出的 `zoom` 脚本示例（可放到 `~/.tmux/zoom`）：

```
#!/bin/bash -f
currentwindow=`tmux list-window | tr '\t' ' ' | sed -n -e '/(active)/s/
^[^:]*: *\([^ ]*\) .*/\1/gp'`;
currentpane=`tmux list-panes | sed -n -e '/(active)/s/^\([^:]*\):.*/\1/
gp'`;
panecount=`tmux list-panes | wc | sed -e 's/^ *//g' -e 's/ .*$//g'`;
inzoom=`echo $currentwindow | sed -n -e '/^zoom/p'`;
if [ $panecount -ne 1 ]; then
    inzoom="";
fi
if [ $inzoom ]; then
    lastpane=`echo $currentwindow | rev | cut -f 1 -d '@' | rev`;
    lastwindow=`echo $currentwindow | cut -f 2- -d '@' | rev | cut -f 2- -d
'@' | rev`;
    tmux select-window -t $lastwindow;
    tmux select-pane -t $lastpane;
    tmux swap-pane -s $currentwindow;
    tmux kill-window -t $currentwindow;
```

```
else
    newwindowname=zoom@$currentwindow@$currentpane;
    tmux new-window -d -n $newwindowname;
    tmux swap-pane -s $newwindowname;
    tmux select-window -t $newwindowname;
```

绑定到按键 `z`：

```
unbind z
bind z run ". ~/.tmux/zoom"
```

8.7. 窗口变为面板：join-pane

当你打开多个窗口后，希望把其中某个窗口合并到当前窗口作为一个面板，可以使用 `join-pane`。在 tmux 中按 `prefix + :` 打开命令行，然后输入（示例）：

```
join-pane -s window01 # 合并名称为window01的窗口的默认（第一个）面板到当前窗口中
join-pane -s window01.1 # .1显式指定了第一个面板，.2就是第二个面板(我本地将面板编号起始值设置为1，默认是0)
```

`join-pane` 也可跨会话合并：`join-pane -s [session_name]:[window].[pane]`，例如 `join-pane -s 2:1.1`。当目标窗口/面板数量为 0 时，窗口关闭；当目标会话的窗口和面板数量为 0 时，会话也会关闭。

补充：`swap-pane` 与 `join-pane` 的语法基本一致。

8.8. 其他配置示例

```
bind m command-prompt "splitw -h 'exec man %%'"   # 绑定m键为在新的panel打开man
# 绑定P键为开启日志功能，如下，面板的输出日志将存储到桌面
bind P pipe-pane -o "cat >>~/Desktop/#W.log" \; display "Toggled logging to ~/Desktop/#W.log"
```

9. 复制模式、Buffer 与系统粘贴板

9.1. 复制模式的基本流程

复制模式常用步骤如下：

1. 输入 `prefix + [` 进入复制模式。
2. 按空格键开始选择，移动光标选中区域。

3. 按回车复制并退出复制模式。
4. 输入 `prefix + ]` 粘贴。

查看当前复制模式的按键风格：

```
tmux show-window-options -g mode-keys # mode-keys emacs
```

默认是 `emacs` 风格。你可以切换到 `vi` 风格：

```
setw -g mode-keys vi # 开启vi风格后，支持vi的C-d、C-u、hjkl等快捷键
```

9.2. 自定义复制与选择快捷键

tmux v2.4 之前可用如下方式把复制模式更贴近 `vi`：

```
bind Escape copy-mode # 绑定esc键为进入复制模式
bind -t vi-copy v begin-selection # 绑定v键为开始选择文本
bind -t vi-copy y copy-selection # 绑定y键为复制选中文本
bind p pasteb # 绑定p键为粘贴文本（p键默认用于进入上一个窗口，不建议覆盖）
```

tmux v2.4 及以上，需要使用 `-T` 与 `-X`：

```
bind -T copy-mode-vi v send-keys -X begin-selection
bind -T copy-mode-vi y send-keys -X copy-selection-and-cancel
```

9.3. Buffer 缓存

tmux 的复制内容会进入 `buffer`（粘贴缓存区），可在会话间共享，但默认与系统粘贴板不互通。默认情况下，buffer 内容独立存在于 tmux 进程中，不会自动进入操作系统的剪贴板。常用命令如下：

```
tmux list-buffers # 展示所有的 buffers
tmux show-buffer [-b buffer-name] # 显示指定的 buffer 内容
tmux choose-buffer # 进入 buffer 选择页面(支持jk上下移动选择，回车选中并粘贴
buffer 内容到面板上)
tmux set-buffer # 设置buffer内容
tmux load-buffer [-b buffer-name] file-path # 从文件中加载文本到buffer缓存
tmux save-buffer [-a] [-b buffer-name] path # 保存tmux的buffer缓存到本地
tmux paste-buffer # 粘贴buffer内容到会话中
tmux delete-buffer [-b buffer-name] # 删除指定名称的buffer
```

```
buffer0007: 11 bytes: "buffer0000:"
buffer0006: 3 bytes: "~/\012"
buffer0005: 12 bytes: "ellar/tmux/2"
buffer0004: 245 bytes: "x/2.5/ b\012◆ /usr/local/Cellar/tmux/2.5/ b\012◆ /usr/local/Cellar/tmux/2.5/ b\012◆
r/tmux/2.5/ b\012◆..."
buffer0003: 16 bytes: "local/Cellar/tmu"
buffer0002: 8 bytes: "ux/2.5/\012"
buffer0001: 14 bytes: "/Cellar/tmux/2"
buffer0000: 18 bytes: "/Cellar/tmux/2.5/\012"
```

图 6　`tmux list-buffers` 示例（截图）

```
(0)  buffer0007: 11 bytes: buffer0000:
(1)  buffer0006: 3 bytes: ~/\012
(2)  buffer0005: 12 bytes: ellar/tmux/2
(3)  buffer0004: 245 bytes: x/2.5/ b\012◆ /usr/local/Cellar/tmux/2.5/ b\012◆ /usr/local/Cellar/tmux/2.5/ b\012◆
(4)  buffer0003: 16 bytes: local/Cellar/tmu
(5)  buffer0002: 8 bytes: ux/2.5/\012
(6)  buffer0001: 14 bytes: /Cellar/tmux/2
(7)  buffer0000: 18 bytes: /Cellar/tmux/2.5/\012
```

图 7　`tmux choose-buffer` 示例（截图）

9.4. Linux：接入系统粘贴板（xclip）

先安装 `xclip`：

```
sudo apt-get install xclip
```

然后配置快捷键，把 tmux buffer 与系统粘贴板互通：

```
# buffer缓存复制到Linux系统粘贴板
bind C-c run " tmux save-buffer - | xclip -i -sel clipboard"
# Linux系统粘贴板内容复制到会话
bind C-v run " tmux set-buffer \"$(xclip -o -sel clipboard)\"; tmux paste-
buffer"
```

9.5. macOS：接入系统粘贴板（pbcopy / pbpaste）

在 macOS 上，tmux 会话里不仅 `pbcopy` / `pbpaste` 可能失效，一些依赖用户会话环境的命令（例如 `osascript`、`open` 等）也可能出现异常。这类问题通常与“命名空间”以及 tmux server 的创建方式有关。

文章中给出了若干相关问题的参考链接（便于排查具体症状）：

- https://apple.stackexchange.com/questions/174779/unable-to-run-display-notification-using-osascript-in-a-tmux-session
- https://stackoverflow.com/questions/30404944/open-command-doesnt-work-properly-inside-tmux/30412054#30412054
- https://stackoverflow.com/questions/16618992/cant-paste-into-macvim/16661806#16661806

解决方案是安装 `reattach-to-user-namespace` 作为包装程序，让 shell 与子进程重新连接到合适的用户级命名空间，从而能访问粘贴板服务（以及相关的用户空间能力）。

安装：

```
brew install reattach-to-user-namespace
```

基础配置：

```
set -g default-command "reattach-to-user-namespace -l $SHELL"
```

如果你在多系统间共享配置文件，tmux v1.9+ 可用 `if-shell` 做条件判断：

```
if-shell 'test "$(uname -s)" = Darwin' 'set-option -g default-command "exec
reattach-to-user-namespace -l $SHELL"'
```

更早版本可用“带检测”的写法：

```
set-option -g default-command 'command -v reattach-to-user-namespace >/dev/
null && exec reattach-to-user-namespace -l "$SHELL" || exec "$SHELL"'
```

然后设置复制/粘贴快捷键（把 buffer 与系统粘贴板互通）：

```
# buffer缓存复制到Mac系统粘贴板
bind C-c run "tmux save-buffer - | reattach-to-user-namespace pbcopy"
# Mac系统粘贴板内容复制到会话
bind C-v run "reattach-to-user-namespace pbpaste | tmux load-buffer - \;
paste-buffer -d"
```

在复制模式中也可以直接把选区送到系统粘贴板：

```
# 绑定y键为复制选中文本到Mac系统粘贴板
bind-key -T copy-mode-vi 'y' send-keys -X copy-pipe-and-cancel 'reattach-
to-user-namespace pbcopy'
# 鼠标拖动选中文本，并复制到Mac系统粘贴板
bind-key -T copy-mode-vi MouseDragEnd1Pane send -X copy-pipe-and-cancel
"pbcopy"
```

完成上述配置后，记得重启 tmux server，使 `default-command` 等设置生效。

10. 会话持久化：Resurrect / Continuum / TPM

关机重启后 tmux 进程退出，会话自然也会消失。如果希望“跨重启保存/恢复会话”，可以使用插件。

10.1. Tmux Resurrect

Resurrect 可保存并恢复窗口、面板顺序、布局、工作目录、运行程序等；其恢复过程具幂等性（不会试图重复恢复已经存在的窗口/面板）。

安装：

```
cd ~/.tmux
mkdir plugins
git clone https://github.com/tmux-plugins/tmux-resurrect.git
```

在 `~/.tmux.conf` 中引入：

```
run-shell ~/.tmux/plugins/tmux-resurrect/resurrect.tmux
```

然后按 `prefix + r` 重载配置。

Resurrect 的常用快捷键：

- 保存：`prefix + Ctrl + s`
- 恢复：`prefix + Ctrl + r`

保存/恢复时，tmux 状态栏会给出提示信息：保存开始会显示类似 “Saving…” 的提示，完成后提示环境已保存；恢复开始会显示 “Restoring…”，完成后提示恢复完成。

保存的数据会写入 `~/.tmux/resurrect`，可按需清理历史备份。

默认情况下，Resurrect 只会恢复一组相对保守的程序列表（例如 `vi vim nvim emacs man less more tail top htop irssi mutt`）。如需恢复更多程序，可参考插件文档中的 “Restoring programs” 说明：

- https://github.com/tmux-plugins/tmux-resurrect/blob/master/docs/restoring_programs.md

如果你之前使用过 tmuxinator，可以考虑迁移到 tmux-resurrect；迁移说明参考：

- https://github.com/tmux-plugins/tmux-resurrect/blob/master/docs/migrating_from_tmuxinator.md#migrating-from-tmuxinator

可选配置示例：

```
set -g @resurrect-save 'S' # 修改保存指令为S
set -g @resurrect-restore 'R' 修改恢复指令为R
# 修改会话数据的保持路径，此处不能使用除了$HOME, $HOSTNAME, ~之外的环境变量
set -g @resurrect-dir '/some/path'
```

进阶备份（需显式开启）的方向包括：

- 恢复 Vim/Neovim 会话（依赖 Vim 的 `vim-obsession` 插件保存会话）。
- 恢复面板内容（pane contents）。
- 保存 shell 历史（实验性功能，且只有“无前台任务运行”的面板其 shell 历史能被保存）。

Vim/Neovim 会话恢复示例：

```
cd ~/.vim/bundle
git clone git://github.com/tpope/vim-obsession.git
vim -u NONE -c "helptags vim-obsession/doc" -c q
```

并在 `~/.tmux.conf` 增加：

```
set -g @resurrect-strategy-vim 'session' # for vim
set -g @resurrect-strategy-nvim 'session' # for neovim
```

恢复面板内容：

```
set -g @resurrect-capture-pane-contents 'on' # 开启恢复面板内容功能
```

注意：使用该功能时请确保 `default-command` 不包含 `&&` 或 `||`，否则可能触发已知问题。你可以用下面的命令查看当前 `default-command`：

```
tmux show -g default-command
```

保存 shell 历史：

```
set -g @resurrect-save-shell-history 'on'
```

10.2. Tmux Continuum

Continuum 在 Resurrect 基础上把“保存/恢复”自动化：默认每隔约 15 分钟保存一次，也可以改成一天一次或关闭。

安装（依赖 Resurrect）：

```
cd ~/.tmux/plugins
git clone https://github.com/tmux-plugins/tmux-continuum.git
```

引入：

```
run-shell ~/.tmux/plugins/tmux-continuum/continuum.tmux
```

常用配置:

```
set -g @continuum-save-interval '1440'
set -g @continuum-save-interval '0'
set -g @continuum-restore 'on' # 启用自动恢复
set -g status-right 'Continuum status: #{continuum_status}'
```

其中 `#{continuum_status}` 是一个插值（format），既可以放在 `status-right` 也可以放在 `status-left` 中，用于展示当前 Continuum 的运行状态。

状态栏可能显示:

```
Continuum status: 1440
Continuum status: off
```

禁用“启动时自动恢复”的方式:

- 移除 `@continuum-restore` 配置。
- 或创建空文件 `~/tmux_no_auto_restore`（文件存在时，自动恢复不会触发）。

Continuum 还支持“开机自动启用 tmux”（macOS 场景更常见）:

```
set -g @continuum-boot 'on'
```

macOS 下可选项包括:

- `set -g @continuum-boot-options 'fullscreen'`
- `set -g @continuum-boot-options 'iterm'`
- `set -g @continuum-boot-options 'iterm,fullscreen'`

Linux 下没有这些选项，通常只能设置为自动启用 tmux server。

10.3. TPM (Tmux Plugin Manager)

当插件变多时，建议使用官方的插件管理器 `tpm` 统一管理（tmux v1.9+）。

安装:

```
cd ~/.tmux/plugins
git clone https://github.com/tmux-plugins/tpm
```

在 `~/.tmux.conf` 中添加（确保初始化行在文件靠后位置）:

```
# 默认需要引入的插件
set -g @plugin 'tmux-plugins/tpm'
set -g @plugin 'tmux-plugins/tmux-sensible'

# 引入其他插件的示例
# set -g @plugin 'github_username/plugin_name' # 格式：github用户名/插件名
# set -g @plugin 'git@github.com/user/plugin' # 格式：git@github插件地址

# 初始化tmux插件管理器(保证这行在~/.tmux.conf的非常靠后的位置)
run '~/.tmux/plugins/tpm/tpm'
```

然后按 `prefix + r` 重载配置使其生效。

基于 tpm：

- 安装插件：在 `~/.tmux.conf` 增加 `set -g @plugin '...'`，然后按 `prefix + I` 下载并刷新环境。
- 更新插件：按 `prefix + U`，选择待更新插件后回车确认。
- 卸载插件：从配置中移除插件行，再按 `prefix + alt + u` 删除插件文件。

11. 会话共享（结对编程 / 远程教学）

Tmux 支持同一会话被多个客户端连接并实时同步，这使得结对编程成为可能。

11.1. 使用 tmate 快速分享

tmate 是 tmux 的“共享增强工具”，能自动生成 SSH 连接地址（一个只读、一个可编辑），便于远程协作。

```
brew install tmate
```

```
tmate
```

（示例：启动后在状态栏看到 SSH URL）

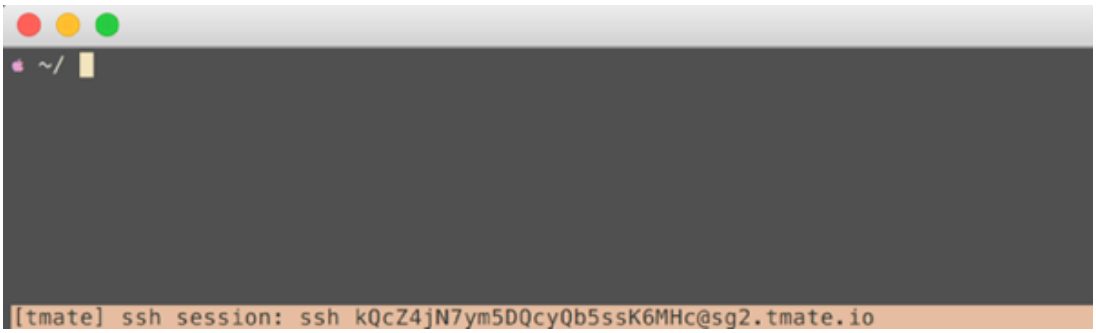

图 8　tmate 显示的 SSH URL（截图）

查看 tmate 生成的地址：

```
tmate show-messages
```

（示例：只读/可编辑两种 URL）

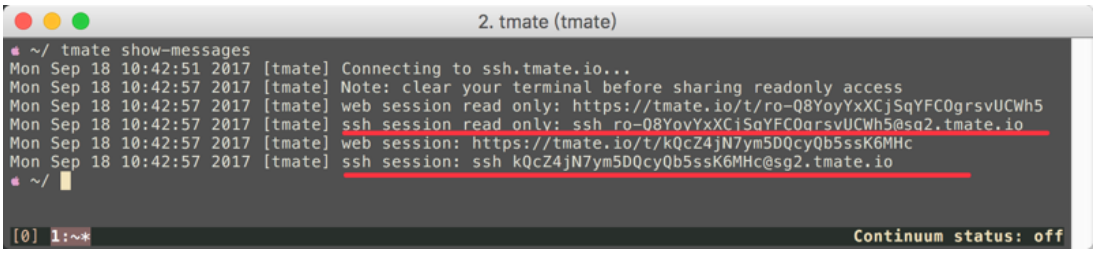

图 9　tmate 生成的两种 URL（截图）

11.2. 组会话（共享账号场景）

如果多方共享同一台服务器账号，且希望减少 tmate 的网络延迟影响，可以用“组会话”：

```
tmux new -s groupSession
```

其他用户通过创建新会话加入该公共会话：

```
tmux new -t groupSession -s otherSession
```

该方式既共享输出，又允许不同用户在不同窗口各自操作，适合结对编程。

11.3. Socket 共享（不同账号场景）

如果不同用户在服务器上对同一目录拥有读写权限（例如 `/var/tmux/`），可使用指定 socket 文件共享会话：

```
tmux -S /var/tmux/sharefile
```

另一个用户接入：

```
tmux -S /var/tmux/sharefile attach
```

注意：使用指定 socket 创建的会话会加载“第一个创建会话的用户”的 `~/.tmux.conf`；后续加入的用户也会沿用同一份配置。

12. 优化与脚本化实践

12.1. 窗口/面板起始编号

```
set -g base-index 1 # 设置窗口的起始下标为1
set -g pane-base-index 1 # 设置面板的起始下标为1
```

12.2. 状态栏优化（示例）

下面配置展示了状态栏的常见可定制项（编码、刷新间隔、对齐、颜色、左右内容、长度、窗口显示格式等）：

```
set -g status-utf8 on # 状态栏支持utf8
set -g status-interval 1 # 状态栏刷新时间
set -g status-justify left # 状态栏列表左对齐
setw -g monitor-activity on # 非当前窗口有内容更新时在状态栏通知

set -g status-bg black # 设置状态栏背景黑色
set -g status-fg yellow # 设置状态栏前景黄色
set -g status-style "bg=black, fg=yellow" # 状态栏前景背景色

set -g status-left "#[bg=#FF661D] ❐ #S " # 状态栏左侧内容
set -g status-right 'Continuum status: #{continuum_status}' # 状态栏右侧内容
set -g status-left-length 300 # 状态栏左边长度300
set -g status-right-length 500 # 状态栏左边长度500

set -wg window-status-format " #I #W " # 状态栏窗口名称格式
set -wg window-status-current-format " #I:#W#F " # 状态栏当前窗口名称格式(#I：序号，#w：窗口名称，#F：间隔符)
set -wg window-status-separator "" # 状态栏窗口名称之间的间隔
set -wg window-status-current-style "bg=red" # 状态栏当前窗口名称的样式
set -wg window-status-last-style "fg=red" # 状态栏最后一个窗口名称的样式
```

```
set -g message-style "bg=#202529, fg=#91A8BA" # 指定消息通知的前景、后景色
```

12.3. 256 colors 支持

在 tmux 中运行 Vim 时配色不一致，通常需要显式开启 256 colors：

```
set -g default-terminal "screen-256color"
```

或：

```
set -g default-terminal "tmux-256color"
```

或启动 tmux 时强制 256 色（示例）：

```
alias tmux='tmux -2' # Force tmux to assume the terminal supports 256
colours
```

12.4. 关闭自动重命名（节省资源）

tmux 默认会自动重命名窗口，频繁命令行操作会触发重命名，可能造成额外 CPU 开销。建议关闭：

```
setw -g automatic-rename off
setw -g allow-rename off
```

12.5. 去掉小圆点（避免窗口被最小尺寸限制）

当同一会话被多个客户端连接时，tmux 可能以最小的窗口尺寸为准，导致其它终端窗口多余空间被“小圆点”填充。解决思路是连接会话时断开其它客户端连接：

```
tmux a -d
```

如果你已在会话内部，也可在命令模式（`prefix + :`）输入 `a -d` 达到同样效果（等价于 `attach -d`）。

```
`: a -d
```

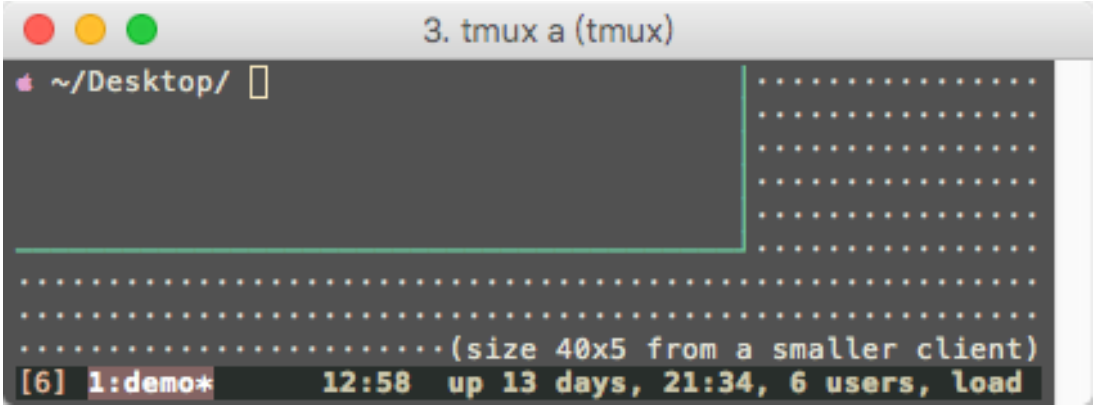

图 10　多客户端尺寸限制导致的“圆点填充”（截图）

12.6. 脚本化创建工作环境（示例）

tmux 支持纯命令行创建会话/窗口/窗格并注入命令，这非常适合批量维护会话列表或“一键恢复本地服务”。脚本场景中通常用 `-d` 后台创建。

```
# 重命名init会话的第一个窗口名称为service
tmux rename-window -t "init:1" service
# 切换到指定目录并运行python服务
tmux send -t "init:service" "cd ~/workspace/language/python/;python2.7
server.py" Enter
# 默认上下分屏
tmux split-window -t "init:service"
# 切换到指定目录并运行node服务
tmux send -t "init:service" 'cd ~/data/node-webserver/;npm start' Enter
```

新建更多窗口并运行程序:

```
# 新建一个名称为tool的窗口
tmux neww -a -n tool -t init # neww等同于new window
# 运行weinre调试工具
tmux send -t "init:tool" "weinre --httpPort 8881 --boundHost -all-" Enter
```

利用 `processes` 选项新建窗口并执行命令（命令结束后窗口关闭；前台命令会让窗口保持到退出）:

```
tmux neww-n processes ls # 新建窗口并执行命令，命令执行结束后窗口将关闭
tmux neww-n processes top # 由于top命令持续在前台运行，因此窗口将保留，直到top命令
退出
```

水平分屏并启动服务（示例）:

```
# 水平分屏
tmux split-window -h -t "init:tool"
# 切换到指定目录并启用aria2 web管理后台
tmux send -t "init:tool" "cd ~/data/tools/AriaNg/dist/;python -m
SimpleHTTPServer 10108" Enter
```

12.7. macOS：开机自动启用脚本

把上面的 tmux 脚本合并为一个可执行脚本（例如 `init.sh`），赋予可执行权限：

```
chmod u+x ./init.sh
```

然后在“系统偏好设置 → 用户与群组 → 登录项”中点击 `+` 添加该脚本，实现开机自动准备工作环境。

图 11　macOS 登录项中添加脚本（截图）

13. 其他命令

```
# 列出所有快捷键，及其对应的 Tmux 命令
$ tmux list-keys
```

```
# 列出所有 Tmux 命令及其参数
$ tmux list-commands

# 列出当前所有 Tmux 会话的信息
$ tmux info

# 重新加载当前的 Tmux 配置
$ tmux source-file ~/.tmux.conf
```

14. Reference

本文整理与撰写过程中参考了如下资料；文中部分示例与截图也来源于这些资料。

- 《Tmux 使用教程》（博客文章）：Tmux 使用教程
- 《tmux：初窥门径》（博客文章）：tmux：初窥门径
- `man tmux`（tmux(1) 手册页）
- tmux Wiki：tmux wiki
- TPM（Tmux Plugin Manager）：tpm
- Tmux Resurrect：tmux-resurrect
- Tmux Continuum：tmux-continuum